

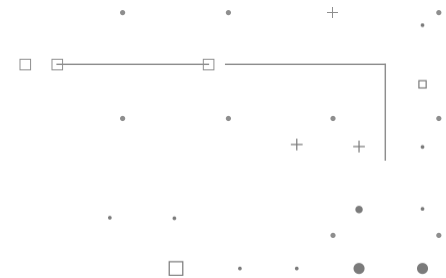
FIAP

NABA

MBA em Engenharia de Dados

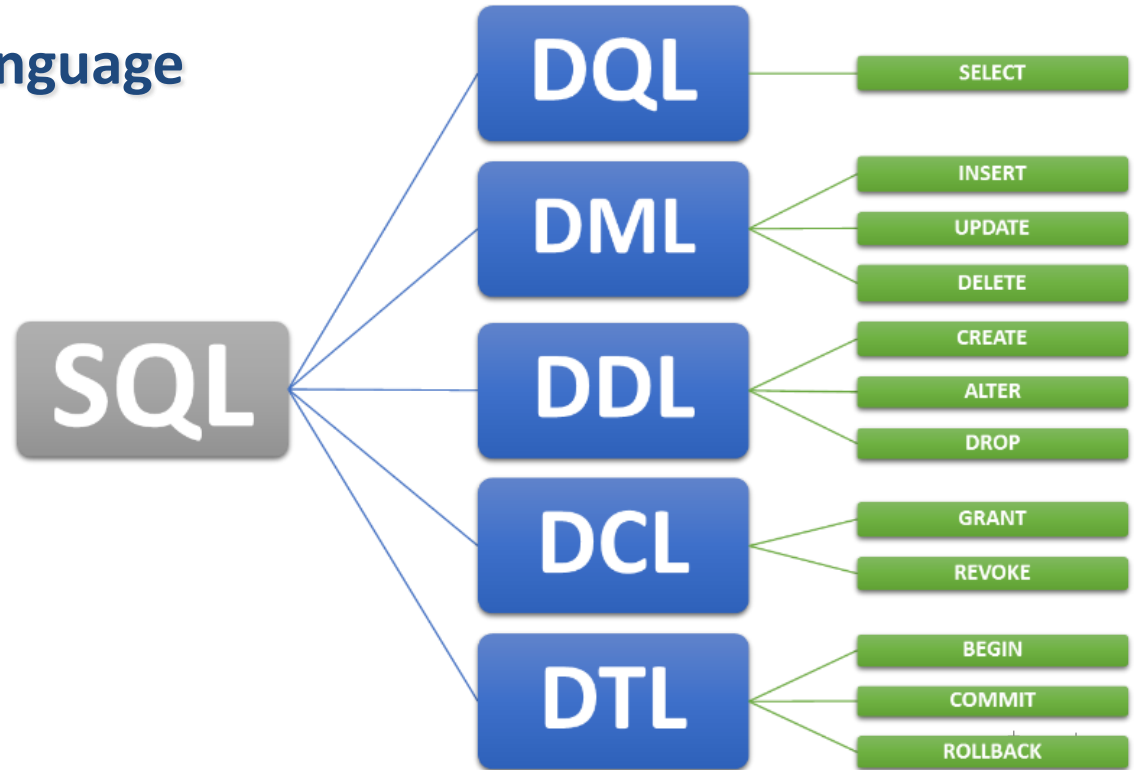


Relational Database Advanced SQL



Structured Query Language

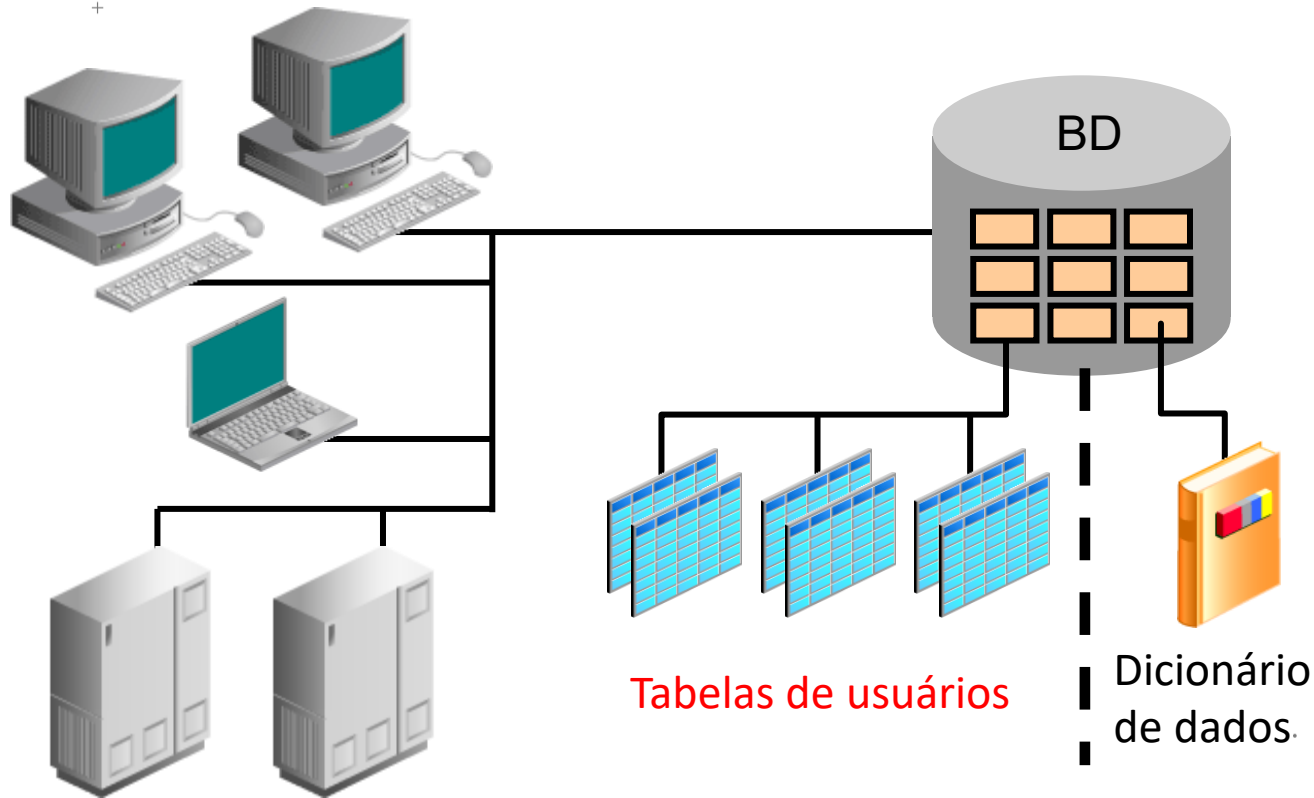
Uma linguagem padrão de gerenciamento de dados que interage com os principais sistemas gerenciadores de bancos de dados baseados nos modelos relacionais. (SGBDR).



SQL – Structured Query Language

Tipo	Instrução	Descrição
DQL ou DRS (Data Query Language ou Data Retrieve Statment)	SELECT	Retorna os dados consultados das tabelas do banco de dados.
DML (Data Manipulation Language)	INSERT UPDATE DELETE MERGE	Insere novas linhas, altera dados em colunas e apaga linhas das tabelas do banco de dados.
DDL (Data Definition Language)	CREATE ALTER DROP RENAME TRUNCATE COMMENT	Cria, altera e remove estruturas dos diversos objetos relacionais.
DTL (Data Transaction Language)	COMMIT ROLLBACK SAVEPOINT	Confirma ou desfaz ações de manipulação de dados.
DCL (Data Control Language)	GRANT REVOKE	Concede ou revoga direitos de acesso ao banco de dados ou a objetos do banco de dados.

Tabelas dos Usuários e Dicionário de Dados



Basic SQL

Comandos SQL Básicos

- Construção e gerenciamento de objetos simples como Heap Tables e índices
- Comandos DML para inserção de uma única linha ou atualizações e remoções simples de dados
- Consultas (SELECT) que acessam tabelas, com seleções, operações de grupos e ordenações simples. Por vezes usando funções SQL básicas.

Advanced SQL



Comandos SQL Avançados

- Construção e gerenciamento de objetos complexos, como tabelas e índices particionados e IOTs (Index Organized Tables)
- Uso de **Subconsultas** e Operadores de Conjunto
- Comandos DML para inserção de múltiplas linhas em um único comando SQL ou atualizações de dados mais complexos (correlatos)
- Consultas (SELECT) com estruturas mais complexas, em geral usadas em sistemas analíticos

Manipulação de Grandes Conjuntos de Dados

- Uma **subconsulta** pode ser utilizada selecionar dados de uma tabela para serem inseridos em outra tabela.
- Ela também pode ser utilizada para na cláusula `WHERE` para realizar operações de atualizações ou de deleções em massa.
- A subconsulta pode ser utilizada na cláusula `FROM` de uma instrução de seleção.

Instruções DML – Data Manipulation Language – EXEMPLOS (AVANÇADO)

INSERT

```
INSERT INTO EMPLOYEES_RETIRED
    (employee_id, first_name, last_name, email,
     phone_number, hire_date, retired_date, job_id,
     salary, commission_pct)
SELECT employee_id, first_name, last_name, email,
       phone_number, hire_date, sysdate, job_id,
       salary, commission_pct
FROM employees
WHERE employee_id=110;
```

Instruções DML – Data Manipulation Language – EXEMPLOS (AVANÇADO)

UPDATE

```
UPDATE employees
SET     department_id = (SELECT department_id
                        FROM employees
                        WHERE employee_id = 130)
WHERE   job_id         = (SELECT job_id
                        FROM employees
                        WHERE employee_id = 114);
```

Instruções DML – Data Manipulation Language – EXEMPLOS (AVANÇADO)

```
UPDATE departments d
SET
d.total_salary = (select sum(e.salary)
                  from employees e
                  where e.department_id = d.department_id)
```

Instruções DML – Data Manipulation Language – EXEMPLOS (AVANÇADO)

DELETE

```
DELETE FROM employees
WHERE department_id =
      (SELECT department_id
       FROM departments
       WHERE department_name = 'Retail Sales');
```

Inserção em múltiplas tabelas

- Uma inserção em múltiplas tabelas é o resultado das linhas retornadas por uma subconsulta executada em uma ou mais tabelas.
- Esse tipo de inserção é muito utilizado em processos de extração, transformação e carga - ETL (*Extraction, Transformation and Loading*)
- Os tipos de inserções em múltiplas tabelas são os seguintes:
 - INSERT Incondicional
 - ALL INSERT Condicional
 - FIRST INSERT Condicional
 - INSERT de Criação de Pivô

Inserção em múltiplas tabelas

- A instrução para a inserção em múltiplas tabelas apresenta a seguinte sintaxe geral a seguir:

```
INSERT [ALL] [clasula_insercao_condicional]  
[clausula_insercao valores] (subconsulta);
```

`clasula_insercao_condicional`

`[ALL] [FIRST]`

`[WHEN condicao THEN] [clausula_insercao valores]`

`[ELSE] [clausula_insercao valores]`

INSERT Incondicional

- Para cada linha retornada pela subconsulta será executada a cláusula `insercao_condicional` para diversas tabelas.

```
INSERT ALL
INTO sal_hist_dept VALUES (EMP_ID, HIREDATE, SALARY)
INTO mgr_hist_dept VALUES (EMP_ID, MANAGER, SALARY)
SELECT employee_id EMP_ID,
       hire_date    HIREDATE,
       salary,
       manager_id   MANAGER
FROM   employees
WHERE  department_id = 90 ;
```

INSERT Condicional: `clasula_insercao_condicional`

- Serão filtradas para cada `clasula_insercao_condicional` pela condição `WHEN` correspondente, que irá determinar se essa cláusula será executada.
- A instrução de inserção pode ser composta por até 127 cláusulas `WHEN`.
- INSERT Condicional:
 - Será avaliada cada cláusula `WHEN` independentemente dos resultados de qualquer outra cláusula `WHEN` avaliada.
 - Cada cláusula `WHEN` que retorna verdade irá executar as cláusulas `INTO` correspondentes.

INSERT Condicional

```
INSERT
  WHEN SALARY < 20000 THEN
    INTO sal_hist_dept
VALUES (EMP_ID, HIREDATE, SALARY)
  WHEN MANAGER IS NULL THEN
    INTO mgr_hist_dept
VALUES (EMP_ID, MANAGER, SALARY)
  SELECT employee_id EMP_ID,
         hire_date   HIREDATE,
         salary,
         manager_id   MANAGER
  FROM employees
 WHERE department_id = 90 ;
```

INSERT Condicional: FIRST

- Será avaliada cada cláusula `WHEN` da instrução. Se a primeira cláusula retornar verdade será executará a cláusula `INTO` correspondente e as cláusulas `WHEN` subsequentes serão ignoradas.

INSERT Condicional: FIRST

```
INSERT FIRST
  WHEN SUM_SALARY > 25000 THEN
    INTO highest_sal VALUES(DEPT_ID, SUM_SALARY)
  WHEN HIREDATE like ('%00%') THEN
    INTO hiredate_hist_00 VALUES(DEPT_ID, HIREDATE)
  WHEN HIREDATE like ('%99%') THEN
    INTO hiredate_hist_99 VALUES(DEPT_ID, HIREDATE)
  ELSE
    INTO hiredate_hist VALUES(DEPT_ID, HIREDATE)
SELECT department_id DEPT_ID,
       SUM(salary)    SUM_SALARY,
       MAX(hire_date) HIREDATE
FROM   employees
GROUP BY department_id;
```

INSERT Condicional: Cláusula ELSE

- Se nenhuma cláusula `WHEN` retornar verdade então:
- Caso tenha sido especificada uma cláusula `ELSE`, será executará a lista de cláusulas `INTO` associada à cláusula `ELSE`.
- Caso não tenha sido especificada uma cláusula `ELSE`, não será executada nenhuma ação para essa linha.

INSERT de Criação de Pivô

- A criação de pivô é uma operação onde será realizada uma transformação de forma que cada linha de uma tabela será convertida em várias outras linhas.

INSERT de Criação de Pivô – Exemplo

```
INSERT ALL
  INTO sales_information
VALUES (emp_id, last_name, week_id, MONDAY)
  INTO sales_information
VALUES (emp_id, last_name, week_id, TUESDAY)
  INTO sales_information
VALUES (emp_id, last_name, week_id, WEDNESDAY)
  INTO sales_information
VALUES (emp_id, last_name, week_id, THURSDAY)
  INTO sales_information
VALUES (emp_id, last_name, week_id, FRIDAY)
SELECT employee_id emp_id,
       last_name name,
       week_id, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY
FROM sales_data;
```

Instrução MERGE

- Essa instrução permite atualizar ou inserir dados em uma tabela de acordo com uma condição.
- Caso a linha exista será executada uma atualização senão será executada uma inserção caso a linha seja nova.

Instrução MERGE - Sintaxe

```
MERGE INTO nome_tabela apelido_tabela
      USING (tabela|visão|sub_consulta) apelido
      ON (condicao_juncao)
      WHEN MATCHED THEN
          UPDATE SET
              coluna1 = coluna_valor1,
              coluna2 = coluna_valor2
      WHEN NOT MATCHED THEN
          INSERT (list_colunas)
VALUES (valores);
```

Instrução MERGE - Exemplo

```
MERGE
  INTO catalog1 s1
  USING catalog2 s2
  ON (s1.id = s2.id)
  WHEN MATCHED THEN
    UPDATE SET s1.price = s2.price
  WHEN NOT MATCHED THEN
    INSERT (id, item, price)
    values (s2.id, s2.item, s2.price);
```

AGRUPAMENTO ROLLUP

- O agrupamento `ROLLUP` resulta em um conjunto com as linhas agrupadas e os valores de subtotais.
- Além dos resultados regulares de agregação retornados pela cláusula `GROUP BY`, o operador `ROLLUP` produz subtotais de grupo .

AGRUPAMENTO ROLLUP-Exemplo

```
SELECT    department_id dept, job_id job,  
          SUM(salary) sum_salary  
FROM      employees  
WHERE     department_id >= 50  
GROUP BY ROLLUP(department_id, job_id);
```

AGRUPAMENTO CUBE

- O agrupamento `CUBE` resulta em um conjunto com a linhas de `ROLLUP` e as linhas de tabelas de referência cruzada. Além dos subtotais gerados pela operador `ROLLUP`, o operador `CUBE` gera subtotais para todas as combinações das dimensões especificadas.

AGRUPAMENTO CUBE – Exemplo

```
SELECT    department_id, job_id job,  
          SUM(salary) sum_salary  
FROM      employees  
WHERE     department_id >= 50  
GROUP BY  CUBE(department_id, job_id);
```

Função GROUPING

Essa função retorna:

1 para indicar que a linha pertence a um determinado grupo e
0 para indicar que a linha não pertence do grupo.



Dica!

Consultas Hierárquicas

- As consultas hierárquicas são utilizadas para recuperar dados baseando-se em um relacionamento hierárquico.
- Uma consulta hierárquica utiliza as cláusulas `CONNECT BY` e `START WITH`

Consultas Hierárquicas - Sintaxe

```
SELECT [LEVEL], coluna, expressao...  
FROM   tabela  
[WHERE condicao]  
[START WITH condicao]  
[CONNECT BY PRIOR condicao]
```

Consultas Hierárquicas - Sintaxe

- A posição inicial da árvore é determinada pela cláusula **START WITH** da seguinte forma:

— `START WITH coluna1 = valor`

- Para percorrer a árvore de cima para baixo deve ser utilizada a cláusula **CONNECT BY PRIOR** da seguinte forma:

— `CONNECT BY PRIOR coluna1 = coluna2`

Consultas Hierárquicas-Exemplos

- **Arvore de hierarquia percorrida de baixo para cima:**

```
SELECT employee_id, last_name, job_id, manager_id
FROM   employees
START  WITH  last_name = 'Kochhar'
CONNECT BY PRIOR manager_id = employee_id ;
```

Consultas Hierárquicas-Exemplos

- **Árvore percorrida de cima para baixo:**

```
SELECT last_name||' Responde para '||  
PRIOR last_name "Árvore Hiede Cima para Baixo",  
LEVEL "Nível"  
FROM employees  
START WITH employee_id = 100  
CONNECT BY PRIOR employee_id = manager_id;
```

CLÁUSULA WITH

- A cláusula `WITH`, apresenta a seguinte sintaxe a seguir

```
WITH nome_da_consulta AS (subconsulta)
    [, nome_da_consulta AS (subconsulta) ]...
```

CLÁUSULA WITH

- Vejamos um exemplo onde a consulta a seguir irá selecionar o nome do departamento e o total de salários dos departamentos quando o somatório de salários dos departamentos é maior que a média dos salários dos departamentos:

CLÁUSULA WITH – Exemplo

- A seleção a seguir será transformada em uma instrução com com a cláusula WITH

```
SELECT      d.department_name, SUM(e.salary) dept_total
FROM        employees e INNER JOIN departments d
ON          e.department_id = d.department_id
GROUP BY    d.department_name
HAVING      SUM(e.salary) > (SELECT AVG(dept_total)
                             FROM (SELECT d.department_name,
                                           SUM(e.salary) dept_total
                                     FROM employees e JOIN departments d
                                     ON e.department_id = d.department_id
                                     GROUP BY d.department_name ))
ORDER BY    department_name;
```


CLÁUSULA WITH – Exemplo

- Após reescrever a consulta anterior com a cláusula `WITH` e ficará da seguinte forma:

```
WITH
    dept_costs AS( SELECT d.department_name,
                          SUM(e.salary) dept_total
                    FROM   employees e JOIN departments d
                          ON e.department_id = d.department_id
                    GROUP BY d.department_name),
    avg_cost AS (SELECT AVG(dept_total)AS dept_avg
                  FROM   dept_costs)

SELECT  *
FROM    dept_costs
WHERE   dept_total >( SELECT dept_avg
                     FROM   avg_cost )

ORDER BY department_name;
```

Instruções DDL – Data Definition Language – EXEMPLOS (AVANÇADO)

Usando subquery (sbu-consultas) no comando CREATE TABLE

```
CREATE TABLE EMPLOYEES_BKP  
AS SELECT * FROM EMPLOYEES;
```

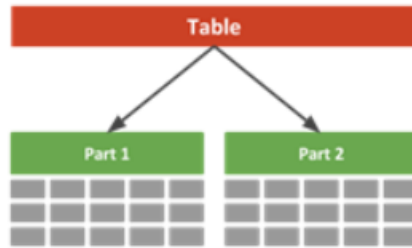
- ➔ Cria a tabela com as colunas retornadas pelo select e carrega os dados
- ➔ Note que a sub-consulta por ser uma consulta complexa, com joins etc

• Tabelas Particionadas

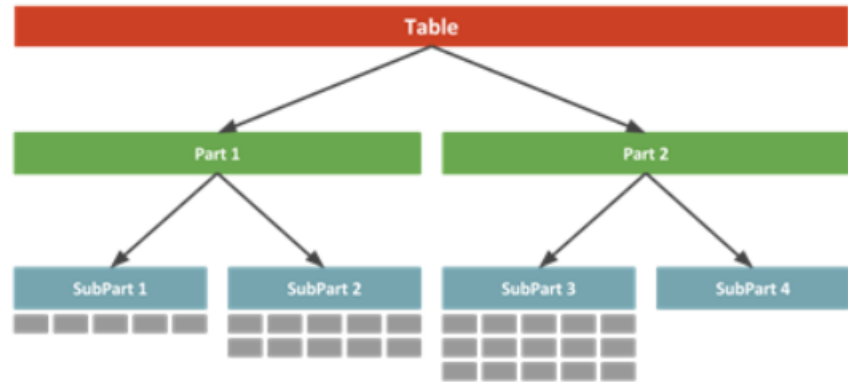
Non-Partitioned



Partitioned



Subpartitioned



- **Tabelas Particionadas**

```
CREATE TABLE sales (  
    sale_id NUMBER,  
    sale_date DATE,  
    amount NUMBER  
)  
PARTITION BY RANGE (sale_date) (  
    PARTITION p_q1_2024 VALUES LESS THAN (TO_DATE('01-APR-2024', 'DD-MON-YYYY')),  
    PARTITION p_q2_2024 VALUES LESS THAN (TO_DATE('01-JUL-2024', 'DD-MON-YYYY')),  
    PARTITION p_q3_2024 VALUES LESS THAN (TO_DATE('01-OCT-2024', 'DD-MON-YYYY')),  
    PARTITION p_q4_2024 VALUES LESS THAN (TO_DATE('01-JAN-2025', 'DD-MON-YYYY'))  
);
```

- **Tabelas Particionadas**

```
CREATE TABLE orders (  
    order_id NUMBER,  
    region VARCHAR2(20),  
    order_date DATE  
)  
PARTITION BY LIST (region) (  
    PARTITION p_north VALUES ('NORTH', 'NORTHEAST'),  
    PARTITION p_south VALUES ('SOUTH', 'SOUTHEAST'),  
    PARTITION p_west VALUES ('WEST'),  
    PARTITION p_other VALUES (DEFAULT)  
);
```

Referências

- Oracle. **Oracle Database Sample Schemas 19c**. Disponível em: <https://docs.oracle.com/en/database/oracle/oracle-database/19/comsc/index.html>. Acesso em: 06 de jul. de 2022.
- Oracle. **SQL Language Reference 12c Release 2 (12.2)**, 2022. Disponível em: <https://docs.oracle.com/en/database/oracle/oracle-database/12.2/sqlrf/index.html>. Acesso em: 06 de jul. de 2022.
- Oracle Education. **Oracle Database 12c: SQL Workshop I/II**. 2013
- Oracle Education. **Oracle Database 12c Administration Workshop Ed 2**, 2014
- PRICE, Jason. **Oracle Database 12c SQL**. Oracle Press, 2013.

OBRIGADO



<https://www.linkedin.com/in/lucianoimelo>

FIAP

Copyright © 2025 | Professores Alexandre Barcelos e Luciano Melo
Todos os direitos reservados. Reprodução ou divulgação total ou parcial deste documento, é expressamente proibido sem consentimento formal, por escrito, do professor/autor.

FIAP